

Lyric Clip Generator

How this app was built

A build-and-decisions log of the work that took this app from an empty repo to a live, paying product — written for the whole team.

Version 1.5 — intelligence, reach & recurring revenue

Repository: github.com/SevrinaLee/lyric-clip-generator

Live: lyric-clip-generator.vercel.app

Prepared: 12 July 2026

Built by SevrinaLee with Claude Code (pair-built).

1. What this app is

Lyric Clip Generator turns a piece of audio into short, vertical, platform-ready lyric video clips. A user uploads a song (or cover, poem, or podcast), adds the lyrics, and the app finds the most hook-worthy moments, scores them, lets the user pick a visual template and preview it, then renders a finished MP4 with burned-in captions ready to post to TikTok, Reels, or Shorts.

It is a full product, not a prototype: real accounts, real per-user data isolation, real video rendering, and real payments (card, PayNow, GrabPay) in Singapore dollars.

The one-line pitch

Audio in - three scroll-stopping, hook-scored lyric clips out, in minutes, with no editing skills required.

How to read this document

Section 2 tells the story in the order it happened, grouped into the phases we actually worked in, with the reasoning behind the important decisions. At the end of each phase you will see the user-journey status as it stood at that moment, so you can watch the app grow. Section 3 is the current status. Appendix A is a plain chronological list of every commit. Appendix B is a technical reference for engineers.

Throughout, each step of the journey is status-coded:

- Live - built, deployed, and working in production.
- Built, needs config - the code is done, but a key or setting is required before it fully works.
- Verify on device - built, but not yet visually confirmed at that screen size.
- Not built yet - does not exist at this stage.

2. The build story, phase by phase

Phase 1 - The core engine (Sprints 1-5)

The first push built the entire spine of the product from an empty repository, following a plan pack (PRD, architecture, sprint breakdown) that shipped with the template. Five sprints landed back to back:

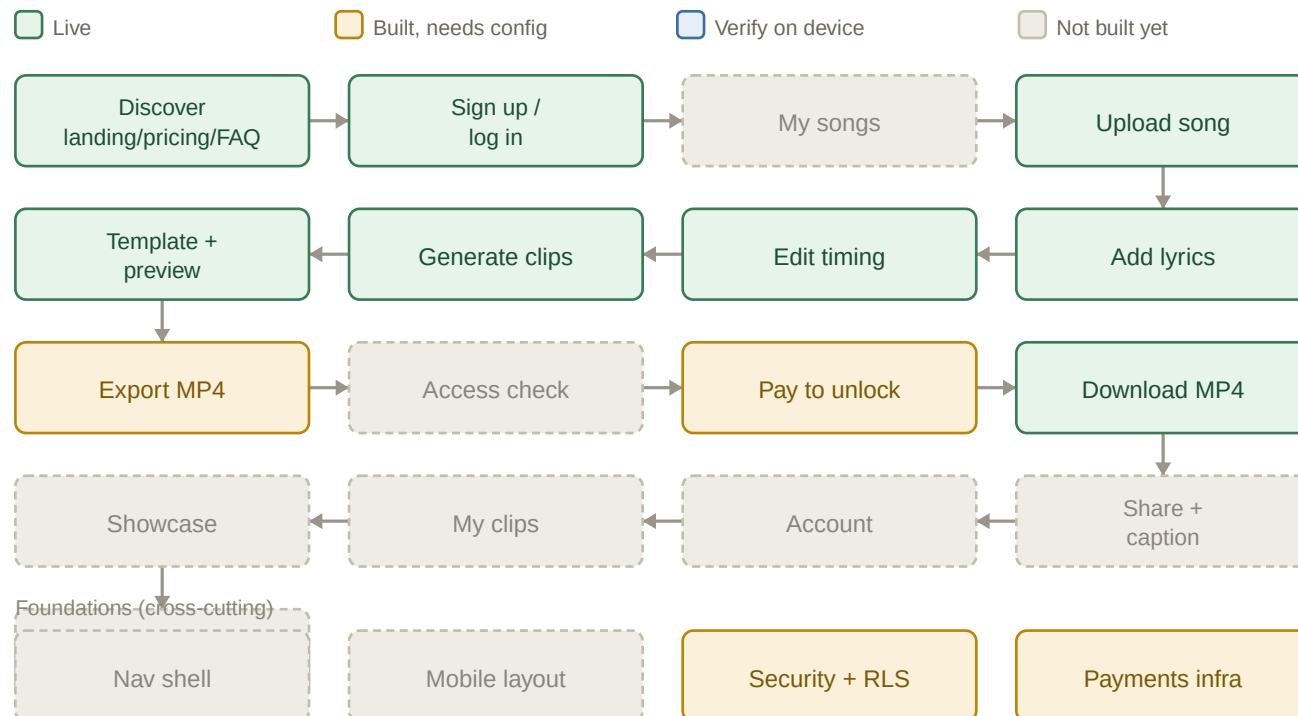
- Sprint 1 - database, song upload, and lyric entry. Audio is stored in Supabase Storage; songs and lyrics live in Postgres.
- Sprint 2 - the heart of the product: rule-based hook scoring (which picks the catchiest moments without needing an AI key), a template picker, and real MP4 export via ffmpeg.
- Sprint 3 - a Stripe checkout payment gate so exports could be monetised.
- Sprint 4 - optional auto-transcription via OpenAI Whisper, for users who do not want to type lyrics.
- Sprint 5 - accounts and a per-user security lock-down using Postgres Row-Level Security (RLS), so one user can never see another's data.

KEY DECISION - WHY RULE-BASED SCORING, NOT AI

The 'intelligence' that picks hook moments is deliberately rule-based, so the core product works with zero AI API keys and zero per-use cost. AI transcription is an optional add-on, never a dependency. This keeps the app cheap to run and resilient.

KEY DECISION - SECURITY IS ENFORCED BY THE DATABASE, NOT THE APP

Data isolation is enforced by Postgres Row-Level Security, so even a bug in application code cannot leak one user's songs or payments to another. The powerful service-role key is only ever used server-to-server, never in the browser.



Journey after Phase 1: the whole create-to-download loop exists. Export is amber because it worked locally but had not yet survived the production environment. Payments are a scaffold with no live key.

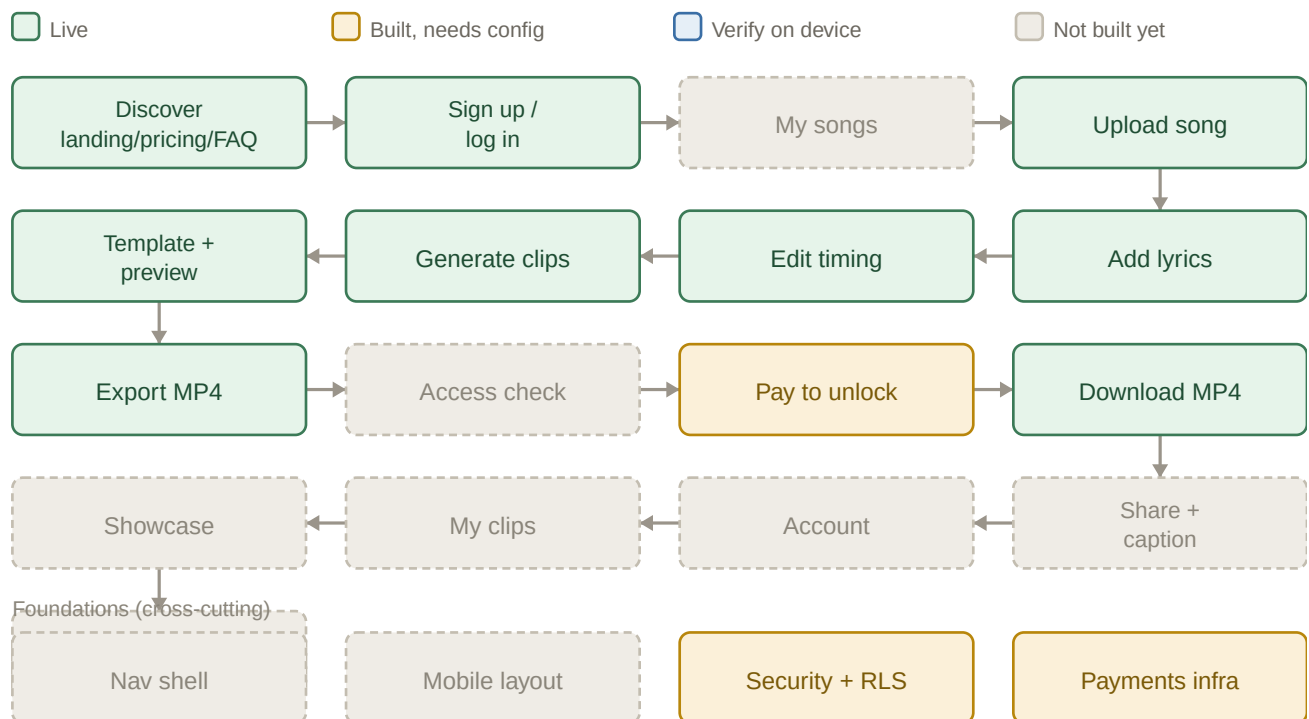
Phase 2 - Making it actually work in production, plus a UX pass

A working demo on a laptop is not a working product. Several things that passed locally broke on the live Vercel/serverless environment, and fixing them was a real chunk of the work — the kind of thing worth documenting because it will recur on any similar project.

- The MP4 export crashed three separate times in production for three different reasons, each invisible on the local machine.
- The template picker, lyric editing, and clip preview were rebuilt to be genuinely usable: edit lyrics with or without timestamps, a live template preview, and a tap-to-time tool that lets you tap along to the song instead of typing timecodes.
- A full brand redesign gave every page a consistent, considered look.

THE PRODUCTION EXPORT SAGA (A CAUTIONARY TALE)

1. The font file path resolved to a numeric module id inside the production bundle, not a real path - crash. Fixed by shipping the font with the app.
 2. The ffmpeg binary was missing from the serverless bundle, and a stale build cache hid it - crash. Fixed by forcing the binary into the bundle.
 3. The production ffmpeg build simply did not include the 'drawtext' feature we relied on - crash. Fixed by switching caption rendering to the 'ass' subtitle method the binary did support.
- Lesson: 'works on my machine' is not 'works in production' - each was only found by reading real production logs.



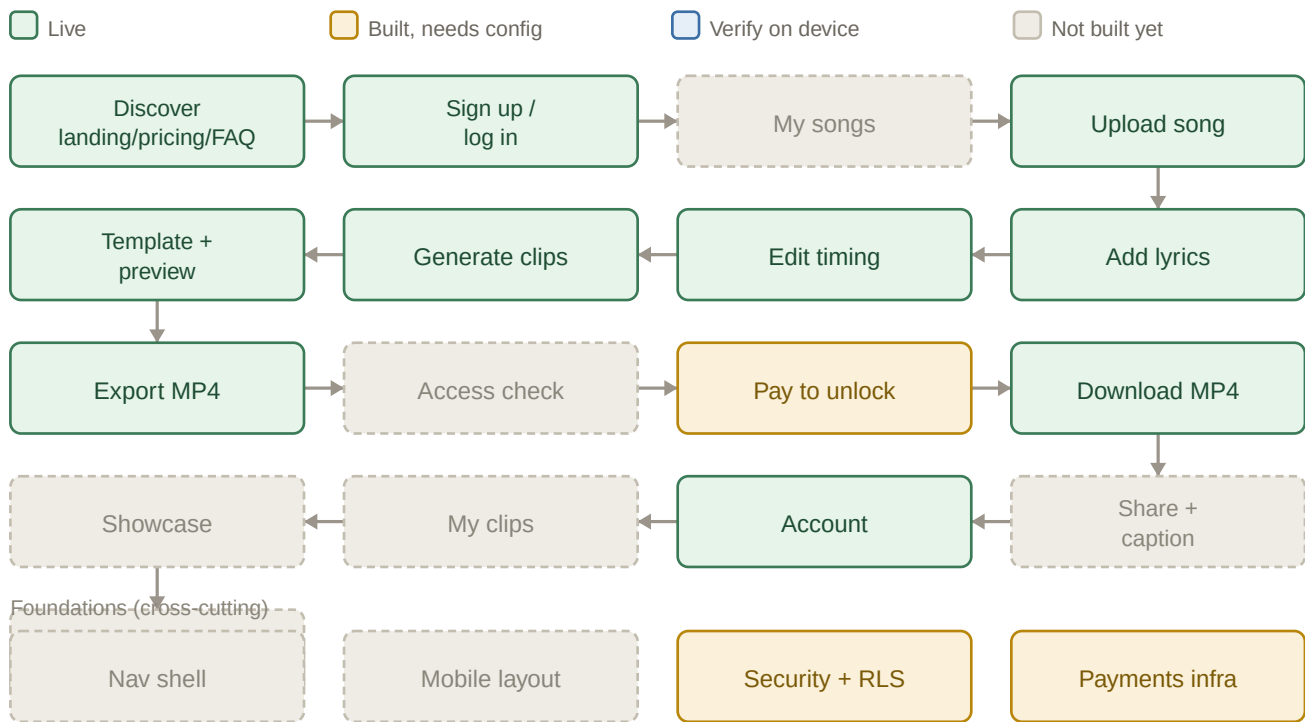
Journey after Phase 2: export is now genuinely live in production. The account area does not exist yet.

Phase 3 - Growth features

With the core solid, the next phase added the things a real product needs around the edges: a forgot-password flow, dedicated Pricing and FAQ pages, more visual templates (including animated gradient backgrounds while keeping captions readable), an account page where users can set a display name and manage billing, and the payments code upgraded to offer multiple methods (card, PayNow QR, GrabPay) in Singapore dollars.

KEY DECISION - PAYMENTS BUILT MODE-AGNOSTIC

Checkout does not hard-code which payment methods to show. It lets the Stripe dashboard decide, so enabling PayNow or GrabPay is a settings change, not a code change. The currency was set to SGD because PayNow and GrabPay require it.



Journey after Phase 3: the account area is live. Payments remain amber - the code is complete, but no live Stripe key was connected yet.

Phase 4 - Security you can prove, and a smarter access model

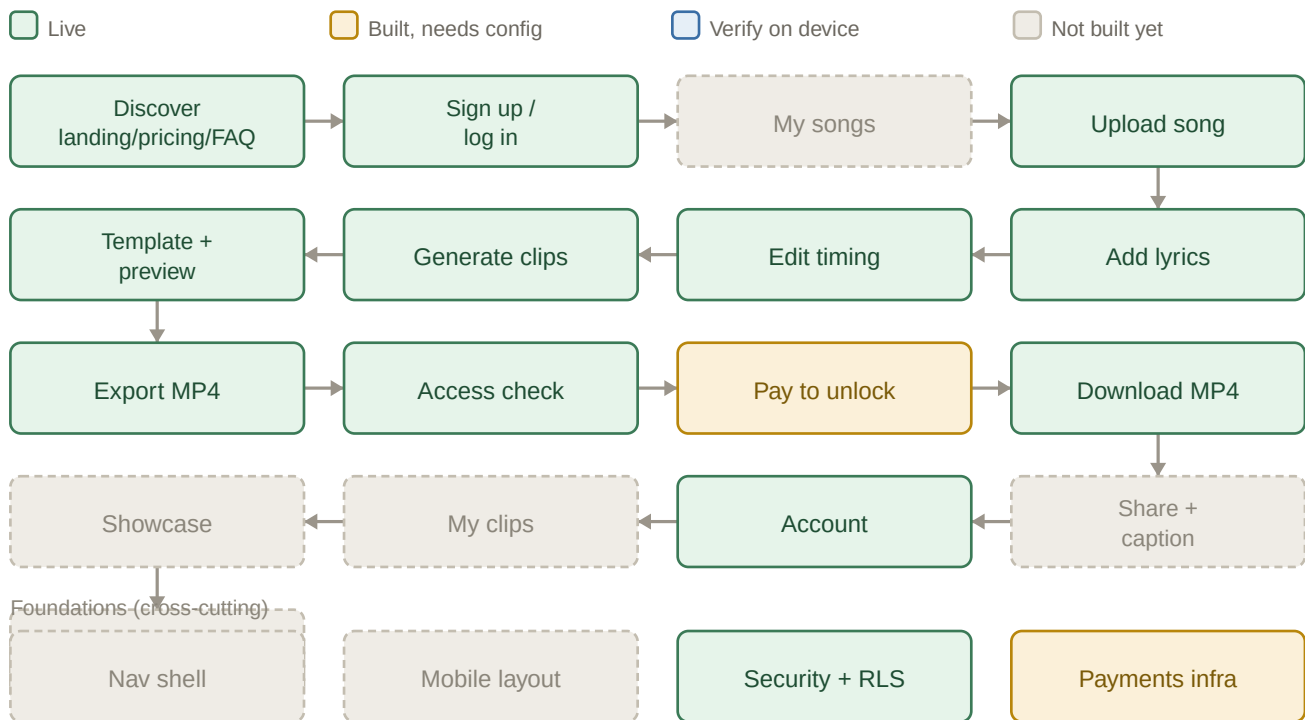
The project's own rules require that any change touching data or auth be backed by automated security tests. This phase delivered that: an executable DevSecOps test suite covering the four things that matter, wired to run automatically on every code change.

- Data isolation - proves one user cannot read or modify another's songs, payments, or profile.
- SQL-injection - fires real attack strings at every input and proves they are treated as harmless text.
- Brute-force - proves rapid failed logins get rate-limited.
- Data-exfiltration - proves list views never leak other users' rows or sensitive fields.

It also introduced a smarter download-access model: founder/QA accounts that get everything free, and one free song for every new user (their first download), with everything after that paid. Both are enforced server-side and locked so a user cannot promote themselves.

KEY DECISION - FOUNDER ACCESS AND THE FREE SONG ARE LOCKED AT THE DATABASE LEVEL

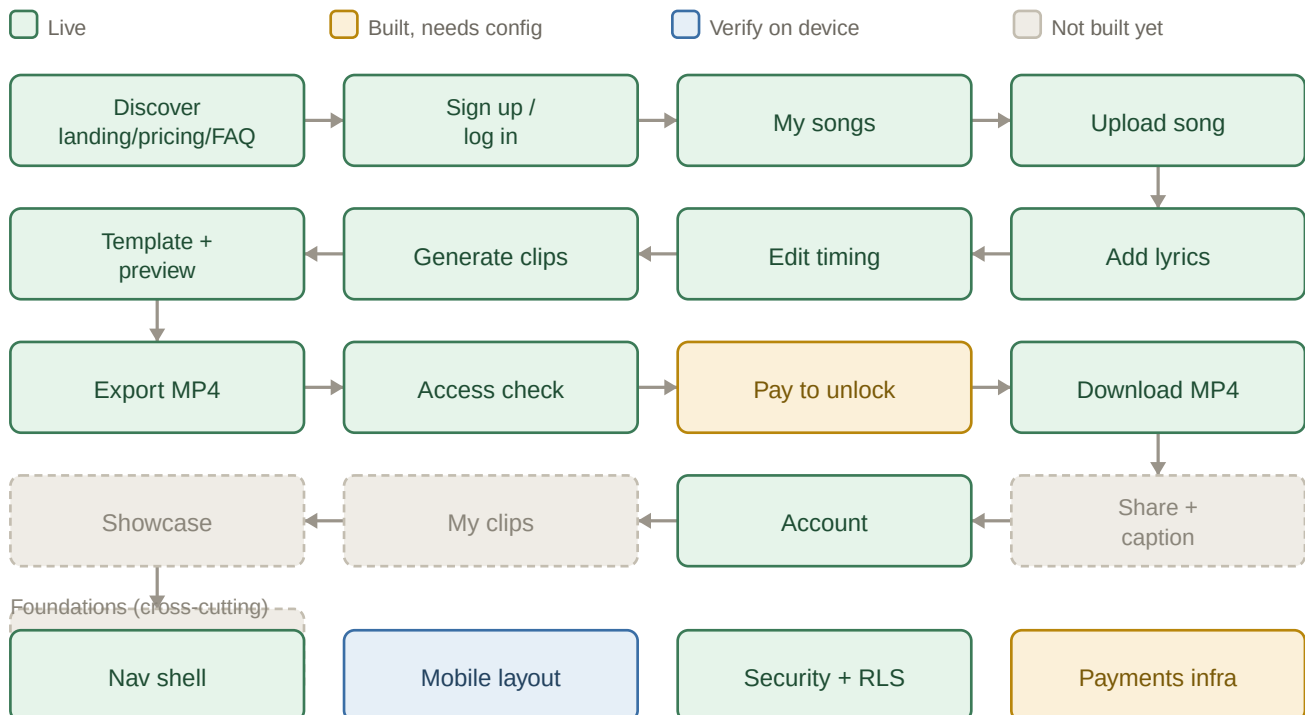
The 'is this account a founder' and 'which song is free' flags can only be written by trusted server code. The database revokes a normal user's ability to change them, so no one can grant themselves free access. A security test guards this forever.



Journey after Phase 4: the access check is live (founder / first-song-free / paid), and security is verified by an automated suite running in CI.

Phase 5 - Navigation shell and mobile

Until now the app had a single top bar. This phase turned it into a proper responsive shell: a left sidebar on desktop that collapses into a hamburger-drawer menu on phones, shown only to signed-in users so the landing page stays welcoming. A new 'My songs' page gave signed-in users a home base listing their own uploads. A follow-up pass tightened spacing, tap targets, and the lyrics editor for small screens.

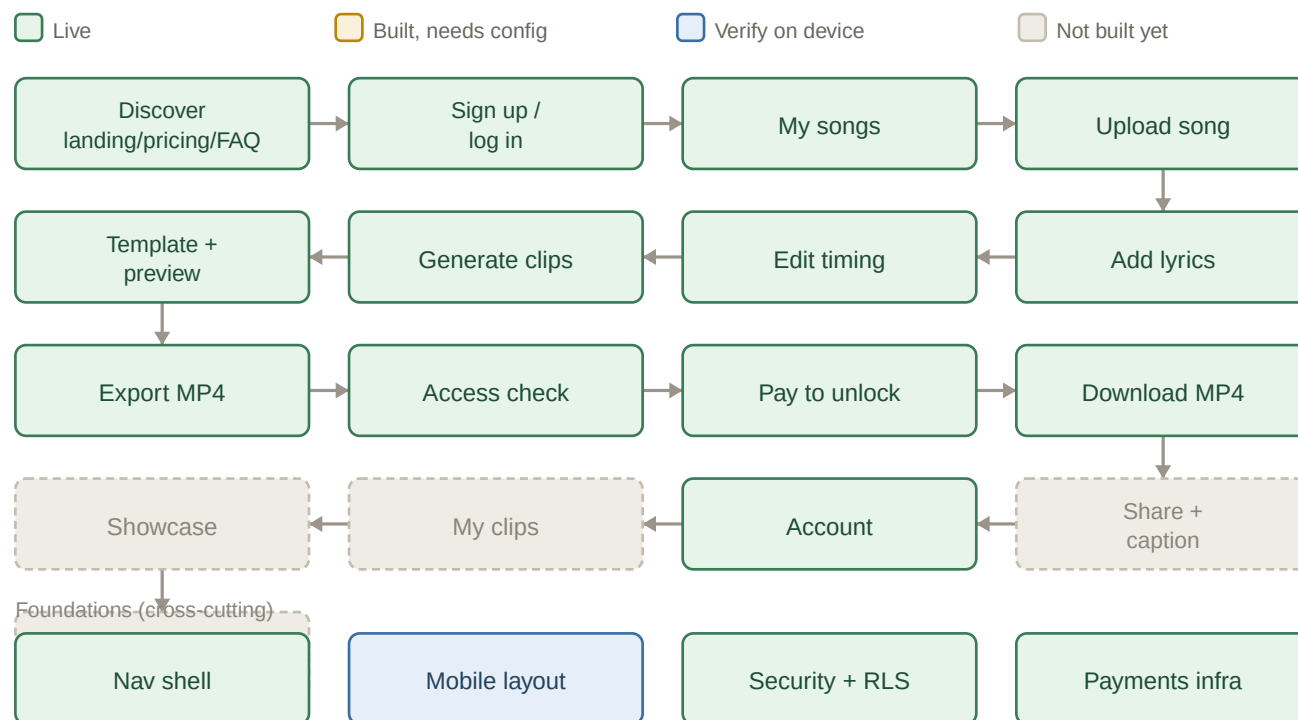


Journey after Phase 5: My songs and the navigation shell are live. Mobile layout is blue - built and code-verified, but the phone-width view had not been driven on a real device in the build environment.

Phase 6 - Payments go live

The final step to a true 1.0: connecting a live Singapore Stripe account. The secret key and webhook were configured, and the whole payment path was verified end to end without spending real money — the live key

was confirmed valid and in live mode, a real checkout session was created and shown to offer card, GrabPay, PayNow and Link, and the webhook was proven to accept correctly-signed events and reject forged ones.



Journey at v1.0: every step of the core loop is live, including real multi-method payments. Only the on-device mobile spot-check remains as blue.

Phase 7 - Expanding the journey (post-1.0)

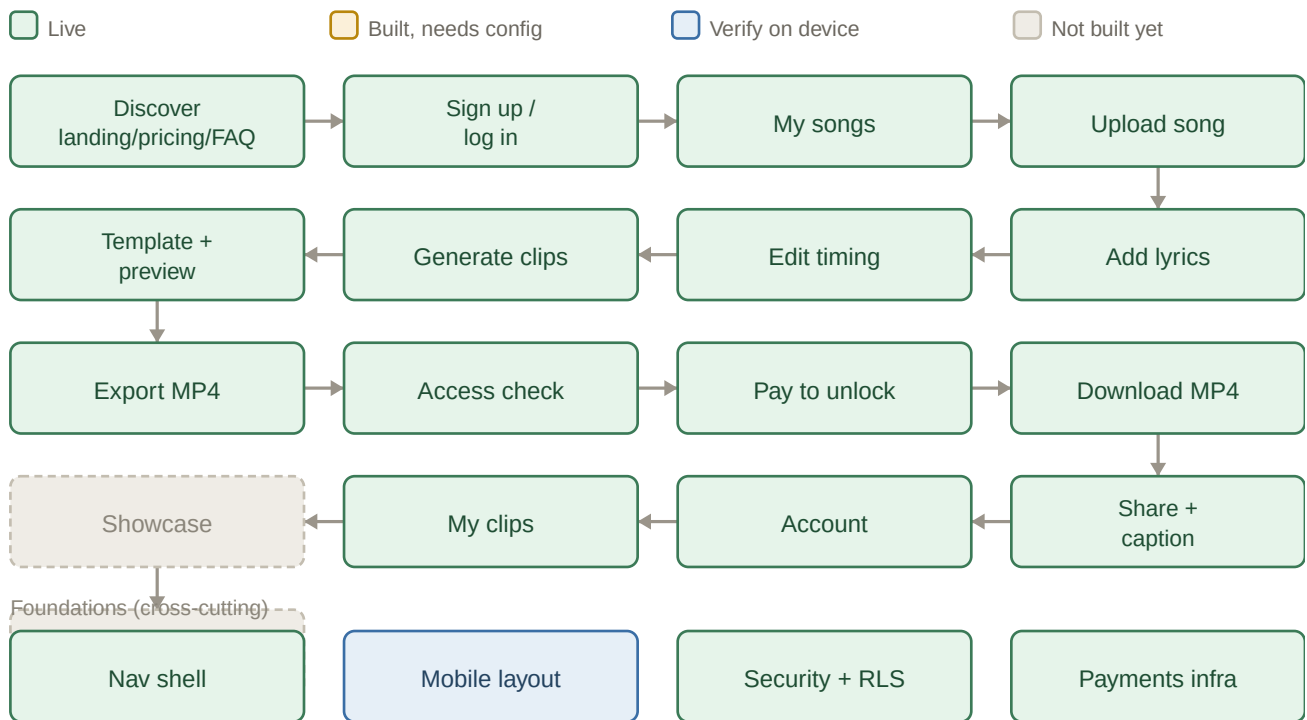
With a shippable 1.0 live, the next phase reshaped the journey from a one-and-done download into a loop, and sharpened the reasons to pay. Three additions, chosen from a brainstorm over the journey diagram itself:

- Retention - a 'My clips' library listing every finished export across songs, so the app becomes somewhere users return to, not a single-use tool.
- Distribution - a share panel offering a ready-to-post caption and platform hashtags (rule-based, no AI key), with copy and native share, carrying users past download toward actually posting.
- Monetization - a value ladder: free exports are watermarked and 720p; paying removes the watermark, renders at full 1080p, and unlocks the premium (gradient) templates.

KEY DECISION - THE WATERMARK IS ALSO THE GROWTH ENGINE

Every free clip carries a small 'made with Lyric Clip Generator' mark, so shared free clips advertise the app. Because rendering happens before payment, the download route lazily re-renders a clean HD version on the first paid download - so paying genuinely removes the mark.

A later refinement closed a trust gap in this loop. A rendered clip is a baked file, so editing lyric timing afterwards left the download silently out of date. Each clip zone now shows a Refresh control that re-renders from the latest saved timing, and when timing changes after a render the zone auto-flags itself 'outdated' (comparing the song's most recent lyric edit to when the clip was rendered) while reassuring the user that the live preview is already current. The result is an explicit guarantee that the downloadable clip can always be brought in line with the latest timing.



Journey after Phase 7: Share + caption and the My clips library are live, and the pay step now drives a watermark / resolution / premium-template value ladder.

Phase 8 - Making clips scroll-stopping (v1.2)

With the loop complete, the clips themselves still looked plain, so a four-sprint push made them customizable and eye-catching. Sprint 1 fixed a live bug - every export had rendered in one fallback font regardless of the template - by vendoring real font files and threading the chosen font and size all the way through, then let users pick per clip. Sprint 2 added the viral-caption toolkit: a high-contrast outlined style, a lower-third position, and word-by-word reveal with a pop (which also fixed a caption animation that had quietly done nothing). Sprint 3 gave backgrounds motion - a gently pulsing gradient, and a premium background that draws the song's own waveform reacting in real time. Sprint 4 gathered every control into one collapsible 'Customize' panel with a reset-to-template.

- Single source of truth - one module resolves a clip's caption style and another resolves its background, and BOTH the live browser preview and the ffmpeg render read the same resolved values, so what you preview is what you download.
- Value ladder preserved - the bold look, outline style, lower third, word-pop, and a pulsing background are free; extra display fonts, the yellow-emphasis style, and audio-reactive waveform backgrounds are premium, all re-checked server-side.
- Verified by frames - because captions are burned in, each new style and background is checked by rendering a real clip and inspecting extracted frames (`npm run verify:captions`), which is how a subtle libass timing bug in the word reveal was caught and fixed.

Phase 9 - Caption intelligence & one-tap Looks (v1.3)

Word-pop looked great but was faked - words appeared on an even split, not the actual singing - because pasted lyrics carry no timing. This phase made it real. When a song is auto-transcribed, Whisper now returns per-word timestamps (stored in a new `lyric_words` table) so word-pop lands on the vocal, and a premium 'karaoke' style fills each word as it's sung. To make all of this reachable in one tap, curated 'Looks' (Viral, Wave, Sunset, Punchy, Minimal) set a clip's template and every caption control at once.

- Foundation-first - Sprint 3.1 shipped the storage and parsing with no behaviour change; Sprint 3.2 then switched the render and preview to consume it, so the risky data work landed separately from the visible feature.
- Honest fallback - lyrics without word timing (pasted, no key) simply use the even split, exactly as before; nothing regresses.

- Security caught a real gap - the automated suite flagged that the first lyric_words insert policy let another user attach words to your lyric; it was hardened before shipping.

Phase 10 - Reach: every format, and creative control (v1.4)

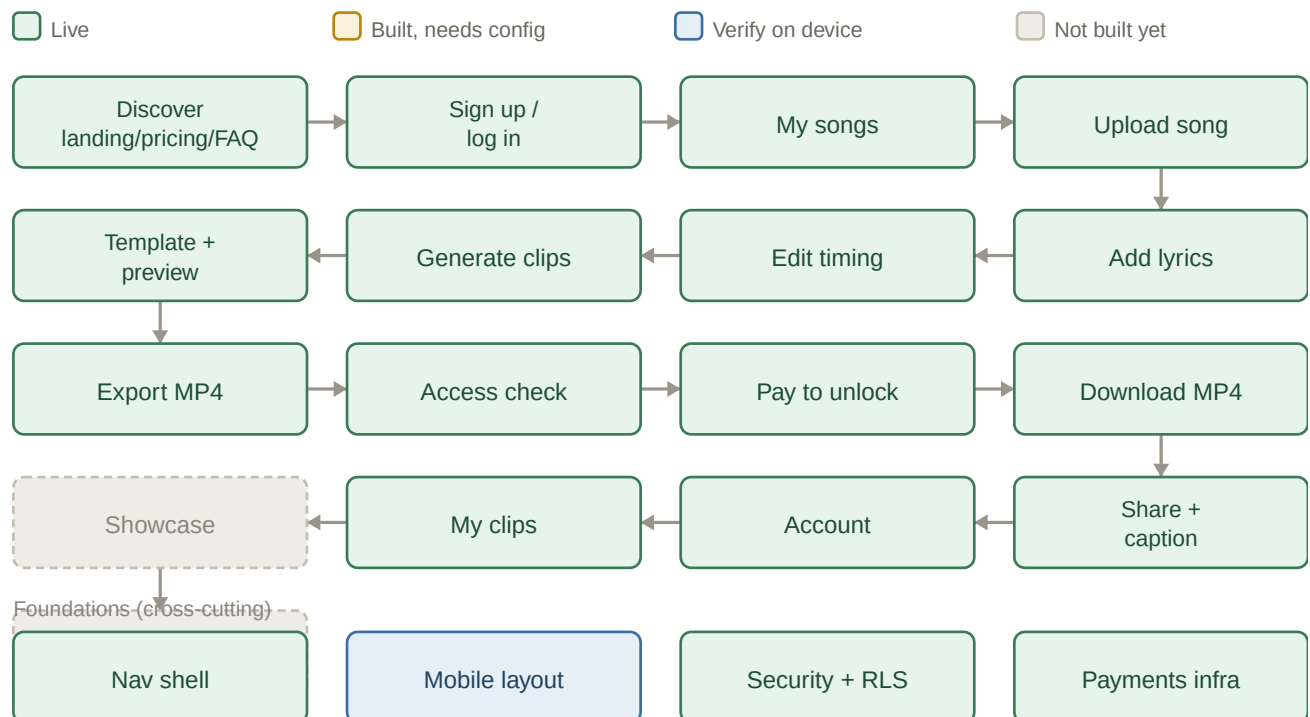
One clip should post everywhere, and the machine's guesses shouldn't be final. Clips now export in 9:16 (free), plus 1:1, 4:5, and 16:9 (premium) - all from the same source, rendered undistorted by authoring captions at a fixed reference height and deriving the width from the chosen aspect. The live preview takes the shape of the selected format, and the clips library tags each export with its ratio. Users can also nudge a clip's start/end by a second at a time (3-60s, within the song) and 'Regenerate' to re-score from the latest lyrics while keeping any clip they've already exported.

- The ladder rides one chokepoint - non-9:16 formats are gated by the same exportTier check that governs every other premium option, so no per-format wiring.
- Adjusting a window marks the download stale (reusing the Refresh flow) and drops the machine 'hook score' - once you've hand-picked the moment, the score isn't ours to assert.

Phase 11 - Recurring revenue & branding (v1.5)

The value ladder gained a top rung and a professional finish. A monthly 'Creator' subscription unlocks every song and all premium options through a single early check in the access resolver - one line lights up fonts, styles, formats, templates, no-watermark and full-HD at once, the payoff of routing the whole ladder through one place from the start. Subscribers also get a brand kit: a custom accent colour (which recolours the caption and waveform), a watermark text, and a logo overlaid on their paid exports.

- Written server-side only - subscriptions and brand kits are owner-read-only tables that only the Stripe webhook / validated actions write, so a client can't grant itself a plan or brand.
- Upload safety - a brand logo is validated by its magic bytes on the server (PNG/JPEG, <=1MB); a renamed .html or an SVG is rejected regardless of its filename.
- One item was attempted and deferred - a background render pipeline (so exports show a progress bar and survive heavy jobs) was built and tested on a preview deploy, but this project's Vercel plan has no Fluid Compute to keep a function alive for post-response work, so renders stay synchronous (reliable, and clips are short) until the plan is upgraded or an external queue is added.

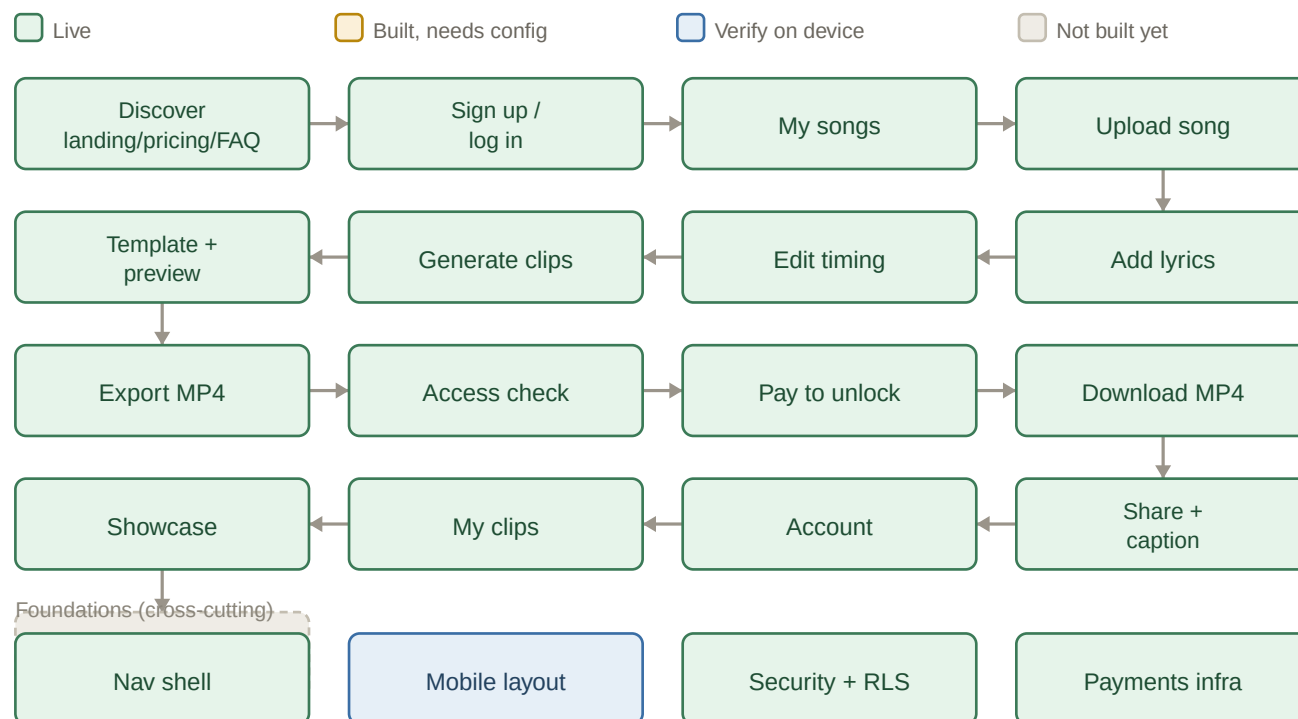


Journey after Phase 11 (v1.5): recurring revenue and brand kits complete the paid ladder. Showcase and Support do not exist yet - both are still 'todo'.

Phase 12 - Growth loop: the public showcase (v1.6)

With the product mature, the next phase gave finished clips somewhere to be seen and to pull new users in. A public showcase gallery displays approved clips with their format-correct aspect, served statically (ISR) via the service-role client because everything shown is intentionally public. A clip only appears after manual approval - there is deliberately no self-approve path, and a per-account cap limits how many submissions can sit pending, so the gallery can't be spammed.

- Approved-only by construction - the `showcase_entries` table has an insert policy that also verifies the submitter owns the underlying export, and NO client update policy, so nobody can flip their own entry to approved. The security suite proves both.
- Zero marginal AI - the whole growth loop is rule-based and static; it costs nothing per view and needs no key.



Journey after Phase 12 (v1.6): the public showcase is live - a discovery surface feeding new users back into the top of the funnel.

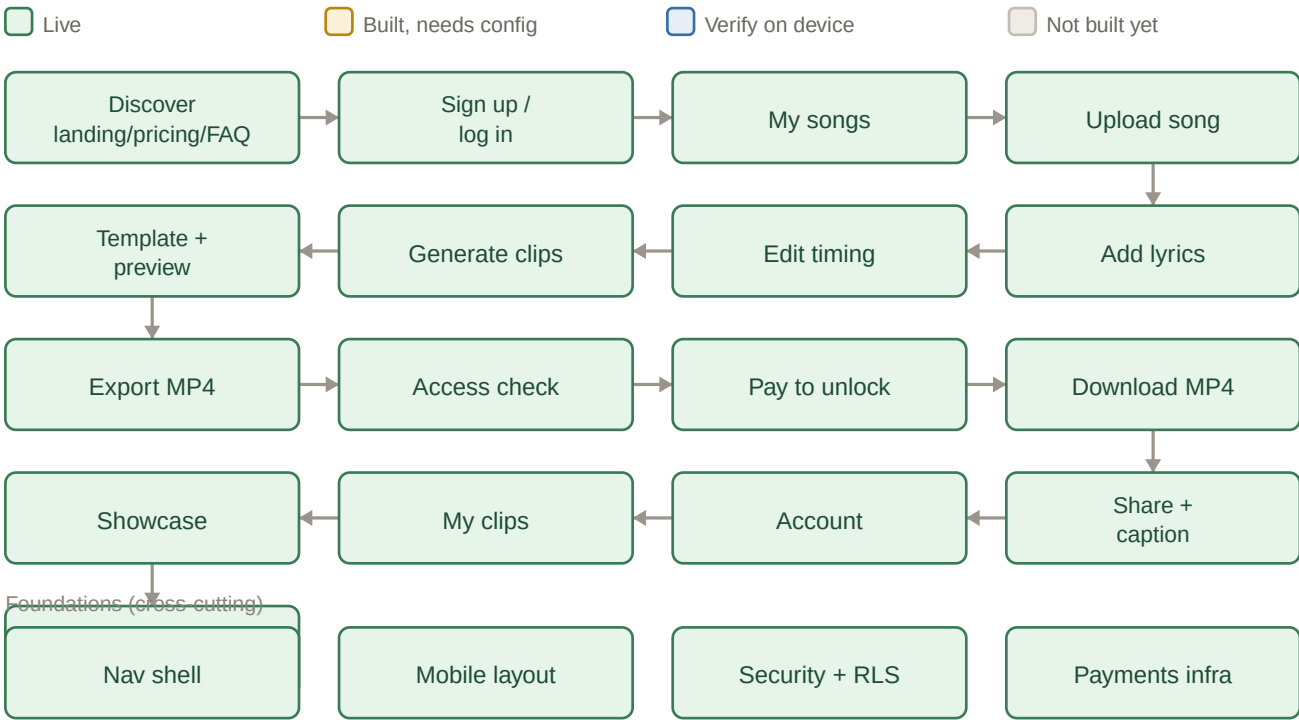
Phase 13 - Creativity & generosity within the free tier (v1.7)

This phase deepened what a creator can make and added a way for happy users to give back - all inside the same free-infrastructure limits (synchronous render, free Supabase, no AI key). A tip jar ('Support') lets anyone leave a one-off donation through Stripe, deliberately decoupled from access so a tip never unlocks a song or grants a plan. Clips gained open-ended custom colours (two background colours plus a caption colour, per clip), a one-tap 'Remix this look' from any showcase card into a new song, a duplicate-a-clip action for fast style variants, and an on-the-fly GIF export (a short, silent, palette-optimised loop). The template and Look libraries also roughly doubled.

- Donation safety - the amount is validated server-side to a sane range and the donation carries no song/payment id, so it can never reach the access resolver; it grants nothing by construction.
- Custom colours are free but strict - every colour is validated server-side AND constrained by a database CHECK to `#rrggbb` hex, so a tampered write can't inject text into the render's filtergraph. A new security section guards it.
- GIF stays inside the budget - it is derived on the fly (never stored), hard-capped to 480px / 15fps / 6s, and gated by the same access check as the MP4 download.

User-uploaded image backgrounds were scoped as Creator-tier and excluded from the public showcase (the one real moderation surface): the image is magic-byte sniffed, stored in a private per-user bucket, and re-encoded on render (stripping EXIF). The showcase gained cursor pagination (a static first page plus a load-

more), and a mobile-viewport pass over every public surface confirmed zero horizontal overflow at phone width - flipping the last 'device' node to live.



Journey at v1.7 (current): the tip jar is live, the showcase feeds a remix loop and paginates, and the mobile layout is verified - every journey node is now live.

3. Where the app stands today (v1.7)

Every step of the core journey works in live production, and the product now spans the full arc from 'make a clip' to 'run a subscription business on it', plus a growth loop and a way to give back: discover (including a public showcase), sign up, upload, auto-transcribe or tap timing, generate and adjust clips, customize captions, backgrounds and per-clip colours, remix a look from the showcase or duplicate a clip, preview in the true export shape, pay per song or subscribe, export in any aspect ratio as MP4 or a short GIF (watermark-free, HD, brand-stamped for subscribers), re-find everything in a clips library, and optionally leave a tip.

Live and working

Discovery (landing plus a public showcase gallery with one-tap remix), accounts and password reset, My songs, upload, lyric entry / timing editors / auto-transcription with per-word timing, rule-based clip generation with window nudging, regenerate and duplicate-a-clip, full per-clip caption customization (font, size, style, position, fade/bounce/word-pop/karaoke) with animated and audio-reactive backgrounds, per-clip custom colours and one-tap Looks, multi-format export (9:16/1:1/4:5/16:9) as MP4 or an on-the-fly GIF, the access gate (founder / first-song-free / single purchase / Creator subscription), real multi-method payments and recurring billing, a Stripe tip jar decoupled from access, the brand kit, the My clips library, share + caption helpers, the account area, the navigation shell, and an automated security suite (extended with a per-clip custom-colours section) running on every change.

Small open items (not blockers)

- Auto-transcribe needs an OpenAI key to switch on. It is the optional path; typing or pasting lyrics works fully without it.
- Mobile layout is verified at phone width (375px) across every public surface with zero horizontal overflow; the signed-in editor uses the same responsive wrapping.
- Two housekeeping items parked by choice: a second founder email still needs to sign up before it can be flagged, and a founder test account is on a temporary password.

THE SINGLE HIGHEST-VALUE NEXT STEP IS DONE

At the time the journey was first mapped, the biggest gap between 'great demo' and 'can take money' was the Stripe key. That is now connected and verified - the app can accept real payments.

Appendix A. Chronological commit log

Every commit, oldest to newest. Read top-to-bottom, this is the same story as Section 2 at finer grain.

Date	What changed and why
2026-06-24	Initialise project, plan pack (PRD, architecture, sprints), initial schema migration
2026-07-08	Restore plan pack + migrations, add storage-buckets migration
2026-07-09	Sprint 1 - demo gallery, song upload, lyric entry
2026-07-09	Sprint 2 - hook scoring, template picker, real MP4 export
2026-07-09	Sprint 3 - Stripe checkout payment gate
2026-07-09	Sprint 4 - Whisper auto-transcription
2026-07-09	Sprint 5 - auth + per-user RLS lock-down
2026-07-09	Fix auth production bugs (site URL, rate limit, hide unconfigured Google button)
2026-07-09	Hide export/edit controls on songs the viewer does not own
2026-07-09	Brand redesign across all pages
2026-07-09	Editable lyrics without timestamps, live template preview, synced clip preview
2026-07-09	Delete lyric lines, correctable timing for all songs
2026-07-09	Tap-to-time lyrics, broader hero copy, account/billing nav
2026-07-09	Fix export crash in production (ERR_INVALID_ARG_TYPE - font path)
2026-07-09	Fix export crash (ffmpeg ENOENT - binary missing from bundle)
2026-07-09	Fix export failing ('Filter not found' - switch to ass subtitles)
2026-07-09	Forgot password, FAQ + pricing pages, gradient background templates
2026-07-09	Multi-method Stripe checkout (SGD/PayNow-ready) + profile display name
2026-07-10	Add DevSecOps security suite (tests/security/)
2026-07-11	Sanitise env secrets (BOM/whitespace) + preflight in security tests
2026-07-11	Founder comp access + one free song per user
2026-07-11	Responsive nav shell + My songs page (Phase 1)
2026-07-11	Mobile optimisation pass (Phase 2)
2026-07-12	Connect and verify live Stripe payments (v1.0)
2026-07-12	Build documentation (PDF + Markdown) from a single-source generator
2026-07-12	My clips history page (retention)
2026-07-12	Share + caption helpers (distribution)
2026-07-12	Watermark + resolution value ladder (monetization)
2026-07-12	Premium templates gated to paid access
2026-07-13	Font fidelity + per-clip font/size customization (aesthetics S1)
2026-07-13	Scroll-stopping caption presets: outline, lower-third, word-pop (S2)
2026-07-13	Animated + audio-reactive waveform backgrounds (S3)
2026-07-13	Consolidated per-clip Customize panel (S4)
2026-07-13	Per-clip Refresh + stale-clip 'outdated' signal
2026-07-14	Clarify first-song-free pricing + prompt caption timing sync (fixes)

2026-07-14	Word-level timestamp foundation (v1.3 S3.1)
2026-07-14	Synced word-pop + premium karaoke captions (S3.2)
2026-07-14	One-tap Looks - curated style bundles (S3.3)
2026-07-15	Multi-format export: 1:1 / 4:5 / 16:9 (v1.4 S4.1)
2026-07-15	Format-aware preview + library, clip-window nudge & regenerate (S4.2/4.3)
2026-07-15	Creator subscription tier (v1.5 S5.2)
2026-07-15	Brand kit: custom watermark, logo, accent (v1.5 S5.3)

Appendix B. Technical reference (for engineers)

Stack

- Next.js 15 (App Router), React 19, TypeScript, Tailwind v4 - hosted on Vercel.
- Supabase - Postgres, Auth, and Storage; sessions via cookies (@supabase/ssr).
- Stripe - Checkout + webhooks, SGD, dashboard-controlled payment methods.
- ffmpeg (via ffmpeg-static) - trims audio and burns in captions using the libass 'ass' subtitle filter; a bundled Noto Sans font is shipped with the app.

How data isolation works

Every user-owned table (songs, lyrics, lyric_words, clip_segments, payments, exports, profiles, subscriptions, brand_kits, render_jobs, showcase_entries) has Row-Level Security enabled with an `auth.uid() = user_id` policy. Adding a new user-scoped table requires an accompanying RLS migration in the same change. Money- and access-granting tables (subscriptions, brand_kits, render_jobs) are owner READ-only - only the service-role client writes them - so a client can never grant itself a plan or fake a render's status. The service-role key bypasses RLS and is used only server-to-server (the Stripe webhook, the free-song claim, and background writes), never in a request carrying a user session.

The access model

Centralised in `lib/access.ts`'s `evaluateSongAccess` - one function every gate consults. A clip's premium options and watermark-free HD download unlock when the account is a founder, OR has an active Creator subscription (which unlocks EVERY song via a single early check), OR the song is the user's one claimed free song, OR a paid payment exists for it. Because every premium feature (fonts, styles, formats, templates, brand kit) routes through this one resolver and the `exportTier` it returns, adding the subscription lit them all up at once. The `is_founder` and `free_song_id` columns are locked at the database grant level so a user cannot self-promote or rotate their free song.

The security suite

`tests/security/` runs on every push/PR via GitHub Actions - across data isolation, SQL-injection prevention, brute-force rate-limiting, data-exfiltration prevention, privilege-escalation prevention, and per-table isolation for every feature table (caption styles, per-word timing, export format, render jobs, subscriptions, brand kits, the public showcase, and per-clip custom colours). It runs against the real Supabase project with disposable users cleaned up after each run. Uploads add a magic-byte file check (`verify:brand`), renders are checked by frame extraction (`verify:captions` / `verify:gif`), and pure validators (donation amounts, custom colours) have their own checks (`verify:donation` / `verify:colors`).

Notable production lessons

- Serverless bundles differ from local: file paths, native binaries, and build caches all bit us on the `ffmpeg` export path.
- The prebuilt `ffmpeg-static` binary lacks the `drawtext` filter; `libass` ('ass') is the reliable way to burn captions.
- Supabase's service-role key in this project carried a byte-order mark; env parsing must strip it or JWTs silently fail.
- Background work (`Next after()` / `waitUntil`) needs Vercel Fluid Compute; without it, post-response callbacks don't finish a ~20s encode - caught on a preview deploy before it could affect production, so rendering stayed synchronous.
- ASS captions authored at a fixed reference height (with an aspect-derived width) render undistorted across every export ratio - no per-format font/margin math.

Regenerating this document

This PDF and its Markdown twin are generated from one source, `docs/build-doc/generate.mjs`. To document a new phase: add a milestone status map and the new content blocks there, then run ``npm install`` (first time) and ``node generate.mjs`` from `docs/build-doc`. Both files regenerate in sync.